

VUE.JS

Aula 2: Reatividade, Formulários e Listas

Capturando eventos, ligando formulários aos dados e exibindo listas dinâmicas

O que vamos ver hoje

1

v-on / @click

Capturando cliques e outros eventos do usuário

2

v-model

Ligando inputs de formulário diretamente aos dados

3

v-for e :key

Renderizando listas dinâmicas

4

methods x computed

Duas formas de calcular valores derivados

5

Prática

Lista de tarefas (to-do list) completa

AULA 2 / 7

Hoje a tela deixa de ser só leitura: o usuário interage e os dados respondem em tempo real.

Revisão rápida da Aula 1

Antes de seguir, lembre o que já sabemos fazer:

data()

Guarda os dados reativos da aplicação

{{ }}

Exibe dados como texto no HTML

v-bind / :

Liga dados a atributos de tags

v-if / v-show

Exibe ou esconde elementos por condição

Hoje: saímos da tela estática e começamos a reagir a cliques, digitação e listas de dados.

v-on: capturando eventos

v-on liga um evento do usuário (clique, digitação, envio) a um método do Vue.

```
// app.js
createApp({
  data() {
    return { contador: 0 };
  },
  methods: {
    incrementar() {
      this.contador++;
    }
  }
}).mount("#app");
```

```
<!-- index.html -->
<p>{{ contador }}</p>

<button v-on:click="incrementar">+1</button>

<!-- forma abreviada (mais usada) -->
<button @click="incrementar">+1</button>
```

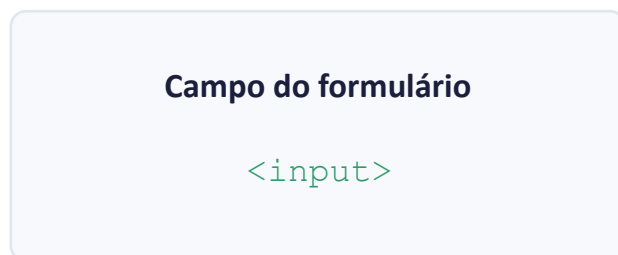
Dentro de methods, **this** se refere aos dados da aplicação — por isso **this.contador++** e não só “contador++”.

v-model: ligando formulários aos dados

v-model cria uma ligação nos dois sentidos: o input atualiza o dado, e o dado atualiza o input.

```
// app.js
createApp({
  data() {
    return { nome: "" };
  }
}).mount("#app");
```

```
<!-- index.html -->
<input v-model="nome" placeholder="Seu nome">
<p>Olá, {{ nome }}!</p>
```



v-model em outros elementos

Funciona também em checkbox, radio e select — mudando o tipo do dado ligado.

```
// app.js
data() {
  return {
    aceito: false,
    corFavorita: "verde"
  };
}
```

O tipo do dado muda conforme o elemento:

checkbox → true / false

select → texto da opção escolhida

```
<!-- checkbox: boolean -->
<input type="checkbox" v-model="aceito">
<p>Aceitou: {{ aceito }}</p>

<!-- select: string -->
<select v-model="corFavorita">
  <option value="verde">Verde</option>
  <option value="azul">Azul</option>
</select>
```

v-for: renderizando listas

v-for repete um elemento HTML para cada item de um array.

```
// app.js
data() {
  return {
    tarefas: ["Estudar Vue", "Fazer exercício", "Revisar HTML"]
  };
}
```

```
<!-- index.html -->
<ul>
  <li v-for="tarefa in tarefas" :key="tarefa">
    {{ tarefa }}
  </li>
</ul>

<!-- com índice -->
<li v-for="(tarefa, index) in tarefas" :key="tarefa">
  {{ index + 1 }} - {{ tarefa }}
</li>
```

Por que o :key importa

O Vue usa a key para identificar cada item da lista de forma única entre atualizações.

Sem :key (evite)

- ✗ Vue pode reaproveitar elementos errados ao reordenar
- ✗ Inputs e estados internos podem “vazar” para o item errado
- ✗ Bugs difíceis de perceber visualmente

Com :key (correto)

- ✓ Cada item mantém sua identidade ao reordenar/remover
- ✓ Use um valor único e estável (id, nunca o índice se a lista muda)
- ✓ Evita bugs de estado trocado entre itens

Regra prática: sempre que a lista tiver um id único, use :key="item.id" em vez do índice.

methods x computed

Duas formas de obter um valor calculado a partir dos dados — com uma diferença importante de performance.

methods

Executa a função toda vez que é chamada na tela, mesmo se nada relevante mudou.

Use para ações (cliques, envio de formulário)

computed

Recalcula só quando um dado que ela usa muda, e guarda o resultado em cache.

Use para valores derivados exibidos na tela

computed na prática

Um computed que conta quantas tarefas ainda faltam concluir.

```
// app.js
createApp({
  data() {
    return {
      tarefas: [
        { texto: "Estudar Vue", feita: true },
        { texto: "Fazer exercício", feita: false },
        { texto: "Revisar HTML", feita: false }
      ]
    };
  },
  computed: {
    restantes() {
      return this.tarefas.filter(t => !t.feita).length;
    }
  }
}).mount("#app");
```

```
<!-- index.html -->
<p>{{ restantes }} tarefa(s)
restante(s)</p>
```

Repare

- restantes não é chamado com parênteses no template
- Ele só recalcula quando o array tarefas muda
- Se fosse um method, recalcularia a cada nova renderização da tela

EXERCÍCIO PRÁTICO

Lista de Tarefas (To-Do List)

O que construir

- Um input com v-model para digitar a nova tarefa
- Um botão que adiciona a tarefa ao array (v-for exibe a lista)
- Cada tarefa deve ter um checkbox v-model para marcar como feita
- Um computed mostrando quantas tarefas restam
- Cada precisa de :key único (use um id, não o índice)

Dica de estrutura

```
data() {  
  return {  
    novaTarefa: "",  
    tarefas: []  
  };  
},  
methods: {  
  adicionar() {  
    this.tarefas.push({  
      id: Date.now(),  
      texto: this.novaTarefa,  
      feita: false  
    });  
    this.novaTarefa = "";  
  }  
}
```

Recapitulando

`v-on / @`

Captura eventos do usuário e chama métodos

`v-model`

Liga um input aos dados nos dois sentidos

`v-for / :key`

Renderiza listas com identidade estável por item

`methods`

Recalcula toda vez que é chamado

`computed`

Recalcula só quando as dependências mudam (com cache)

Próxima aula: Componentização (props, emits e comunicação entre componentes)